7.4.2 Rendering TopoJSON

Because TopoJSON stores its data in a format different from the GeoJSON structure that's expected by `d3.geo.path()`, we need to include Topojson.js and use it to process TopoJSON data to produce GeoJSON features. This is rather straightforward and can be done in a call to our new datafile, as shown in the following listing. Figure 7.16 shows the properly formatted features in your console.

Listing 7.16 Loading TopoJSON

```
queue()
    .defer(d3.json, "world.topojson")
    .defer(d3.csv, "cities.csv")
    .await(function(error, file1, file2) { createMap(file1, file2); });

  function createMap(file1, file2) {

    var worldFeatures = topojson.feature(file1, file1.objects.countries)
    console.log(worldFeatures);
  };
```

Notice that our TopoJSON file has a property "objects", which all TopoJSON files have, but "countries" is specific to this file and might be "rivers" or "land" or other property names in other files.

Now that it's in the format we want, we can send it to our existing code and draw this array of features like we did with the features we loaded from world.geojson. We replace our earlier `countries` with the `worldFeatures` variable declared in listing 7.16. That's all that most people do with TopoJSON, and they're happy for it because TopoJSON data is significantly smaller than GeoJSON data. But because we know the topology of the features in a TopoJSON data file, we do interesting geographic tricks with it.



Figure 7.16 TopoJSON data formatted using `Topojson.feature()`. The data is an array of objects, and it represents geometry as an array of coordinates like the features that come out of a GeoJSON file.